PreCap: Bounded, Self-Normalizing Activations for Deep Normalization-Free MLPs

Depth Calibration, a Comparison with TAT, and 4-Bit Robustness

Aaron Schmitt

Independent researcher, Wiesbaden, Germany · ORCID 0009-0009-8689-5848

Preprint, version 0.2 · 29 October 2026 · Code and data: github.com/hermi-labs/precap-activation

Abstract. PreCap is a smooth, monotonic activation that is bounded on both sides; SPreCap adds SELU-style constants and an input gain b that sets the mean-field gradient gain χ_1. Version 0.1 showed that SPreCap-cs matches SELU in 50- and 100-layer MLPs without normalization or skip connections. Version 0.2 reports seven further preregistered studies (about 2,200 runs, MNIST and FashionMNIST, paired Wilcoxon tests with Holm correction). (1) *Depth calibration.* Setting $\chi_1 = 1 + k/L$ through b trains 100- and 200-layer MLPs clearly better than SELU in all 8 primary cells (15 of 15 seeds each). Both the principle of making activations more linear with depth (Deep Kernel Shaping, Tailored Activation Transformations (TAT)) and the rule of keeping the SELU gradient gain below $1 + 1/L$ are known; our contribution is only a bounded, self-normalizing realization. (2) *Comparison with TAT.* With Gaussian initialization, the depth-calibrated variant kL2 is worse than TAT (Leaky ReLU, official parameters) in 8 of 8 cells. With TAT's own recipe (scaled orthogonal initialization and per-example input normalization), it is on par with TAT in fp32: 2 cells better, 1 worse, 5 without significant difference. Most of the gap came from the recipe, not from the activation. (3) *4-bit robustness.* The bounded member SPreCap-cs is more robust than SELU to post-training 4-bit activation quantization at depth 100. With max calibration the advantage is large (6–19 pp); with the best of four calibrations per run it shrinks to about 2 pp, and at depth 50 the result is mixed. Against TAT, cs is better at 4 bits in 4 of 4 cells with symmetric per-tensor quantization (−1.4 to −7.7 pp) and in 3 of 4 with asymmetric quantization (−1.1 to −2.2 pp), while TAT is clearly better in fp32 (+0.8 to +3.9 pp). No single member of the family has both properties: depth calibration widens the output range and gives up the 4-bit robustness. All negative results are reported.

1 Introduction

Self-normalizing networks [1] showed that an activation function alone can keep the forward signal of deep fully connected networks at zero mean and unit variance. SELU achieves this with an unbounded positive branch. Bounded activations promise numerical safety, for example at low precision, and in deep networks without normalization layers, but classic bounded functions such as tanh saturate and are hard to train at depth [9]. Recent work replaces normalization layers with bounded point-wise functions (DyT [6], Derf [7], SALU [5]).

This work started from a simple idea: use the expressiveness of exponential growth as long as it is harmless, and cap it before anything explodes. It asks a narrow question. *Can a bounded, smooth exponential-type activation be made self-normalizing and match SELU in very deep MLPs without normalization?* Our contributions are:

- the PreCap/SPreCap family and its constants, including a third parameter that sets χ_1 (Section 2);
- a preregistered, hash-frozen evaluation with separate tuning and confirmation seeds, including all negative intermediate results (Sections 4–5);
- evidence that, for this bounded family, test error falls monotonically as $\chi_1 \to 1$, an independent confirmation of the slightly super-critical design in bSELU [2];
- fused forward/backward kernels for CUDA and Apple MLX and a pip-installable package (Section 6).

Version 0.1 ended with three open directions: low-precision inference, normalization-free deep networks, and a depth-dependent $\chi_1 = 1 + k/L$. Version 0.2 follows them and asks three questions. *Does a depth-dependent χ_1 make the bounded family train at depths of 100 and 200? How does it compare with the strongest known method for this setting, TAT [12]? Does the bound pay off when activations are quantized to 4 bits?* The answers are yes, on par under the same recipe, and yes with clear limits. The new contributions of version 0.2 are:

- the depth family kLk with $\chi_1 = 1 + k/L$, solved exactly for $L = 50, 100, 200$ (Section 2.1);

- a preregistered comparison with TAT, first with each method's own recipe and then with the same recipe (Section 5.3);
- a preregistered quantization study with fair calibration, symmetric and asymmetric schemes, and SELU, ELU and TAT as baselines (Section 5.4).

2 Definition

CapExp. For a threshold c, CapExp is exponential below c and continues along its tangent above it:

$$\mathrm{capexp}(w) = e^w - 1 \quad \text{for } w \le c, \qquad \mathrm{capexp}(w) = e^c - 1 + e^c\,(w - c) \quad \text{for } w > c$$

CapExp is exactly $e^c \cdot \mathrm{ELU}(w-c) + e^c - 1$ (verified numerically to $4 \cdot 10^{-15}$). With biases and a following linear layer it therefore represents the same functions as ELU [8]. The new element is the soft clip in front of it.

PreCap and SPreCap. With the clip $w = s \cdot \tanh(bx/s)$, fixed $s = 1.5$ and $c = 1$:

$$f(x) = \lambda \cdot \{1 \text{ if } x > 0;\ \ \alpha \text{ if } x \le 0\} \cdot \mathrm{capexp}(s \tanh(bx/s))$$

PreCap0 is $\lambda = \alpha = b = 1$. It has slope 1 at the origin and is close to $2 \cdot \mathrm{SiLU}$ for small inputs. For SPreCap, (λ, α) are chosen as in SELU so that $\mathrm{E}[f(z)] = 0$ and $\mathrm{E}[f(z)^2] = 1$ for $z \sim \mathrm{N}(0, 1)$, computed with Gauss–Hermite quadrature. The gain b sets $\chi_1 = \mathrm{E}[f'(z)^2]$, which controls how gradients grow from the output to the input layer at initialization: $\log_{10}(g_1/g_L) \approx (L-1)/2 \cdot \log_{10} \chi_1$ [3, 9]. Since tanh is bounded, the output is bounded on both sides (Table 1). The function is monotonic and continuous, its derivative jumps at 0 by the factor α as in SELU, and its second derivative is discontinuous at $w = c$.

Variant	λ	α	b	χ_1	Range
SPreCap	0.80676869644	2.39695130922	1	1.1305	[−1.50, 2.48]
SPreCap-cs	1.102702500	2.053776267	0.71188112	1.0716 (= SELU)	[−1.76, 3.39]
SPreCap-e03	1.985552225	1.608136397	0.4118896262	1.0300	[−2.48, 6.11]
PreCap0	1	1	1	–	[−0.78, 3.08]

Table 1. Constants ($s = 1.5$, $c = 1$). Derivations: `math/p1_spreccap_exact.py`, `math/p3a3_chi1_family.py`.

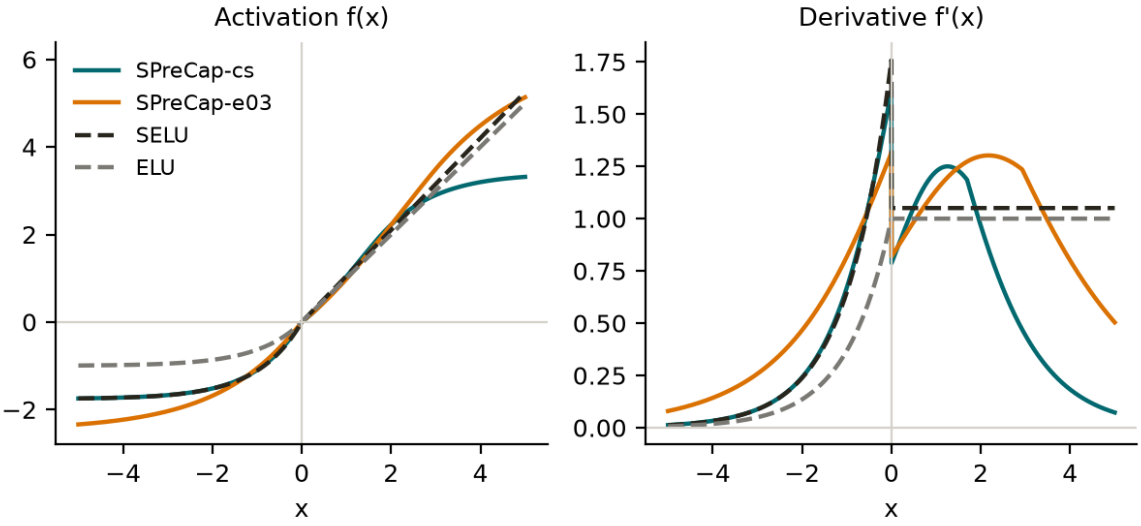


Figure 1. SPreCap-cs and SPreCap-e03 compared with SELU and ELU. Unlike SELU and ELU, the PreCap variants saturate for large positive inputs.

2.1 Depth family kL*k*

At initialization, gradients grow from the output to the input layer roughly like $\chi_1^{(L-1)/2}$ [3]. For SELU and SPreCap-cs ($\chi_1 = 1.0716$) this is about 35× at depth 100 and about 970× at depth 200. Setting $\chi_1 = 1 + k/L$ keeps the growth roughly at $e^{k/2}$, independent of depth (1.65× for $k = 1$, 2.7× for $k = 2$). For each depth, (λ, α, b) are solved jointly so that $\mathrm{E}[f(z)] = 0$, $\mathrm{E}[f(z)^2] = 1$ and $\mathrm{E}[f'(z)^2] = 1 + k/L$ for $z \sim \mathrm{N}(0, 1)$, with $s = 1.5$ and $c = 1$ unchanged (Table 4). kL1 at depth $2L$ uses the same constants as kL2 at depth L. The depth rule itself is not new: for SELU, Chen et al. [17] already recommend a gradient gain $1 + \varepsilon$ with $\varepsilon < 1/L$, so that $(1 + \varepsilon)^L$ stays bounded. kL1 sits exactly at this bound and kL2 at twice the bound. What the third parameter b adds is that the bounded

function can meet the rule exactly at every depth.

Variant	Depth	χ_1	λ	α	b	Range
kL2	50	1.040	1.6141	1.7397	0.4962	[−2.18, 4.97]
kL2 / kL1	100 / 50	1.020	2.6496	1.4613	0.3188	[−3.01, 8.15]
kL2 / kL1	200 / 100	1.010	4.1705	1.2968	0.2124	[−4.20, 12.83]
kL1	200	1.005	6.2966	1.1978	0.1457	[−5.86, 19.38]
SPreCap-cs (fixed)	all	1.0716	1.1027	2.0538	0.7119	[−1.76, 3.39]

Table 4. Constants of the depth family ($s = 1.5$, $c = 1$).

As χ_1 approaches 1, b falls and λ grows, so the bound moves outward. The family therefore trades gradient preservation against a narrow output range. This trade-off decides the quantization results in Section 5.4.

3 Related work

Self-normalization. SELU [1] introduced the mean/variance fixed point. Chen et al. [2] generalize it with explicit tolerances, $\mathrm{E}[f] = \varepsilon_\mu$, $\mathrm{E}[f^2] = 1 - \varepsilon_q$, $\varphi(1) = 1 + \varepsilon_\varphi$, and report that bSELU with a slightly super-critical gradient term beats SELU on a 56-layer CNN without shortcuts (88.54 % vs. 84.81 % on CIFAR-10). Our χ_1 family follows the same idea for a bounded function. Lu et al. [4] obtain bidirectional self-normalization with Gauss–Poincaré normalized activations and orthogonal weights. **Bounded and normalization-free.** DyT [6] and Derf [7] replace normalization layers in Transformers with $\tanh(\alpha x)$ and $\mathrm{erf}(\alpha x + s)$. SaluNet [5] uses the bounded, learnable $\mathrm{SALU}(x) = ax/\sqrt{(1 + abx^2)}$ to train ResNets and Transformers without normalization. **Signal propagation.** Edge-of-chaos theory [3] links $\chi_1 = 1$ to trainable depth. Bukva et al. [9] show that saturation can impede training even at criticality, which matches our observation that the critical point of PreCap0 is fragile. **Similar shapes.** TanhExp, $x \cdot \tanh(e^x)$ [10], combines tanh and exp in the opposite order and is unbounded. A survey of over 400 activations [11] contains no identical function. To our knowledge the exact PreCap form is new, but every building block is known. The contribution is therefore small and specific.

Depth-dependent activation shaping. Deep Kernel Shaping (DKS) [13] and Tailored Activation Transformations (TAT) [12] transform activations so that the kernel of the whole network stays well-behaved at a given depth; TAT does this for Leaky ReLU, whose negative slope grows towards 1 with depth. Shaped activations with slope $1 + c/\sqrt{n}$ [14] follow the same principle in a width-dependent limit. For self-normalizing networks, Chen et al. [17] derive SELU constants with a gradient gain $1 + \varepsilon$ and recommend $\varepsilon < 1/L$. The depth family kLk is an instance of this known principle, not a new idea. What differs is the bounded, self-normalizing form, in which a single constant b sets χ_1 exactly while the mean/variance fixed point is kept. **Quantization.** Post-training quantization with per-channel weights, per-tensor activations, and percentile or MSE-based clipping is standard practice [15]. Clipping activations during training, as in ReLU6 or PACT [16], is a known way to make networks easier to quantize. Our bound acts similarly, but it is part of the activation and is not learned.

4 Experimental protocol

Every study was preregistered before it ran: hypotheses, tests, decision rules, and SHA-256 hashes of all scripts. Hashes were verified before and after each run. Tuning seeds and confirmation seeds are disjoint across all studies. Comparisons are paired by seed and use an exact sign-flip permutation version of the Wilcoxon signed-rank test, which does not depend on the scipy version. Holm correction is applied per test family.

Phase 3a-3 (main study). MLP, width 256, depth 50 and 100, no normalization, no residual connections; MNIST and FashionMNIST, standardized inputs, the last 5000 training images as a fixed validation split; Adam, batch 128, 1720 steps; weights $W \sim \mathrm{N}(0, g^2/\mathrm{fan_in})$ with a variance-preserving gain $g = 1/\sqrt{\mathrm{E}[f(z)^2]}$ for every activation (≈ 1 for SELU and SPreCap). Learning rates $\{1, 2, 5\} \cdot 10^{-5}$, $\{1, 2, 5\} \cdot 10^{-4}$; the rate with the highest median validation accuracy over 4 tuning seeds is selected per activation and cell, then evaluated on 15 confirmation seeds. Activations: SPreCap, SPreCap-cs, SPreCap-e03, SPreCap-cs with a learnable clip s per layer, SELU, and ELU. In total 936 runs. All runs were finite, and the analysis was reproduced independently from the raw data.

Primary hypothesis and v0.1 decision rule. SPreCap-cs vs. SELU in four cells, Holm over 4. SPreCap-cs counts as validated if it is significantly worse in no cell and the median difference is at most +0.5 percentage points in at least 3 of 4 cells. Secondary tests (24, one Holm family) include e03 vs. SELU and cs vs. ELU.

Studies for version 0.2. Seven studies (A, A2, A3, Q1, Q1b, Q2, Q3) were preregistered and hash-frozen before they ran, with one change per study relative to its predecessor. All ran with Apple MLX on an M1 Pro. MLP, width 256, no normalization, no residual connections; Adam, batch 128, MNIST and FashionMNIST, the last 5000 training images as validation split. Learning rates were tuned per activation and cell on 4 tuning seeds over the grid $\{1, 2, 5\}\cdot 10^{-5}$, $\{1, 2, 5\}\cdot 10^{-4}$, or taken over from an earlier study with the identical setup, and then evaluated on 15 disjoint confirmation seeds (10 seeds for the longer budget). Comparisons are paired by seed and use the two-sided Wilcoxon signed-rank test with Holm correction per family ($\alpha = 0.05$). A cell counts as better or worse only if its Holm-adjusted p is below 0.05; otherwise it is inconclusive. Training is bit-reproducible on this platform: re-running the 240 runs of Q1 in Q1b gave 960 of 960 identical values, and the control runs of A2, A3, Q2 and Q3 reproduced earlier runs with a deviation of 0.00 pp.

TAT baseline. Leaky ReLU $\varphi(x) = s\cdot(\max(x, 0) + a\cdot\min(x, 0))$ with $s = \sqrt{(2/(1 + a^2))}$ and the target $\eta = 0.9$, with parameters taken from the official `dks` library [12] (depth 100: $a = 0.5704$; depth 200: $a = 0.6796$) or found by bisection of the composed C-map (depth 50: $a = 0.4305$). Following the TAT recipe, TAT uses scaled orthogonal initialization (SUO) and per-example normalization of the input to mean square 1. We train TAT with Adam instead of K-FAC, so that all methods share one optimizer. The TAT paper reports weaker results with Adam, so our TAT baseline is likely not at its best.

Quantization. Simulated post-training quantization. Weights are int8 per output channel, symmetric. The inputs of every linear layer, including the first, are quantized per tensor to int4 (w8a4) or int8 (w8a8), calibrated on the first 1000 training images. Q1 calibrates with the maximum. From Q1b on, the primary measure is w8a4_best: per run, the calibration with the smallest error on the calibration images among max, 99.9th percentile, 99.99th percentile and MSE-optimal clipping. Q3 adds an asymmetric per-tensor scheme with a zero point.

5 Results

5.1 Version 0.1: SPreCap-cs against SELU (Phase 3a-3)

Cell (data, depth)	SPreCap	SPreCap-cs	cs, learn. *s*	SPreCap-e03	SELU	ELU
FashionMNIST, 50	16.15	14.92	14.83	**14.57**	14.69	14.67
FashionMNIST, 100	24.21	16.98	17.02	**16.61**	17.29	16.62
MNIST, 50	5.13	4.42	4.01	4.05	4.31	**3.75**
MNIST, 100	13.56	6.27	5.92	**5.28**	6.36	5.78

Table 2. Median test error (%) over 15 confirmation seeds at the selected learning rate. Best per row in bold.

Cell	cs − SELU (pp)	cs better / worse	Holm p	e03 − SELU (pp)	Holm p
FashionMNIST, 50	+0.20	4 / 11	0.333	+0.16	1.000
FashionMNIST, 100	-0.12	9 / 6	1.000	-0.74	0.123
MNIST, 50	+0.12	7 / 8	1.000	-0.37	0.360
MNIST, 100	+0.02	7 / 8	1.000	-1.33	**0.0064**

Table 3. Paired tests. Differences are medians of per-seed differences, negative means lower error than SELU. Left: primary family (Holm over 4). Right: secondary family (Holm over 24).

Primary result. SPreCap-cs is significantly worse than SELU in none of the four cells, and all median differences are within ±0.2 pp, so the preregistered v0.1 rule is met. It is not better than SELU either: all four primary tests are inconclusive. In the secondary family SPreCap-cs is better than the original SPreCap in 4 of 4 cells and worse than ELU on MNIST at depth 50 (+0.58 pp, Holm p = 0.022). SPreCap-e03 is better than SELU on MNIST at depth 100 (-1.33 pp, 13 of 15 seeds) and better than SPreCap-cs there. Mean ranks over the four cells: e03 1.5, ELU 1.75, learnable *s* 3.25, cs 4.25, SELU 4.25, SPreCap 6.0.

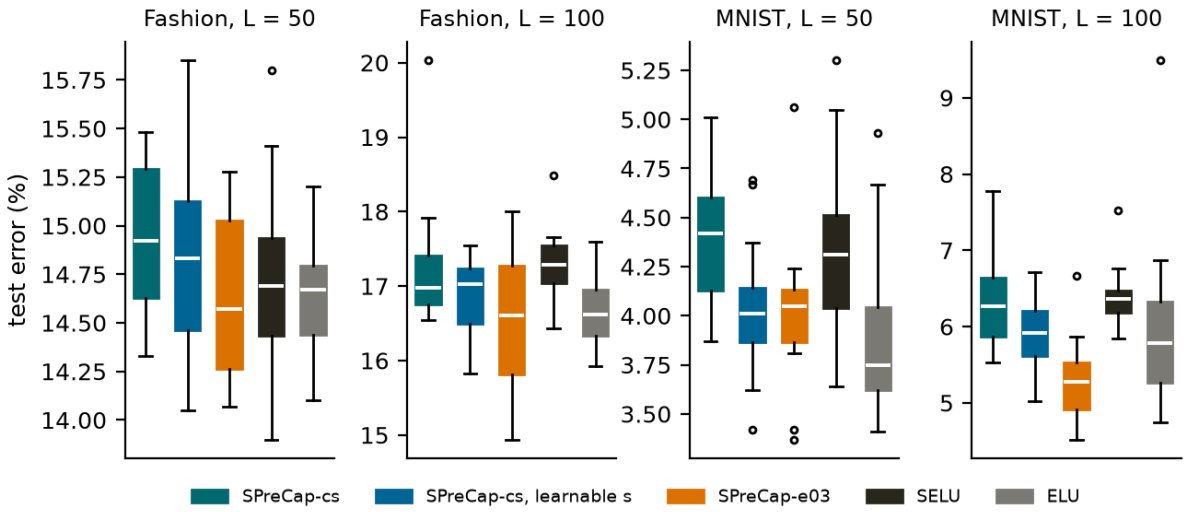


Figure 2. Test error per confirmation seed (15 seeds) at the selected learning rate. The original SPreCap ($\chi_1 = 1.13$) lies far above and is omitted, see Figure 3.

χ_1 dose-response. Within the family, the median error decreases strictly as χ_1 goes from 1.13 to 1.07 to 1.03 in all four cells (Figure 3). The largest gain comes from 1.13 to 1.07. Measured gradient growth at initialization agrees with the mean-field prediction to within 0.17 orders of magnitude (Figure 4), and SPreCap-cs is indistinguishable from SELU at initialization by construction.

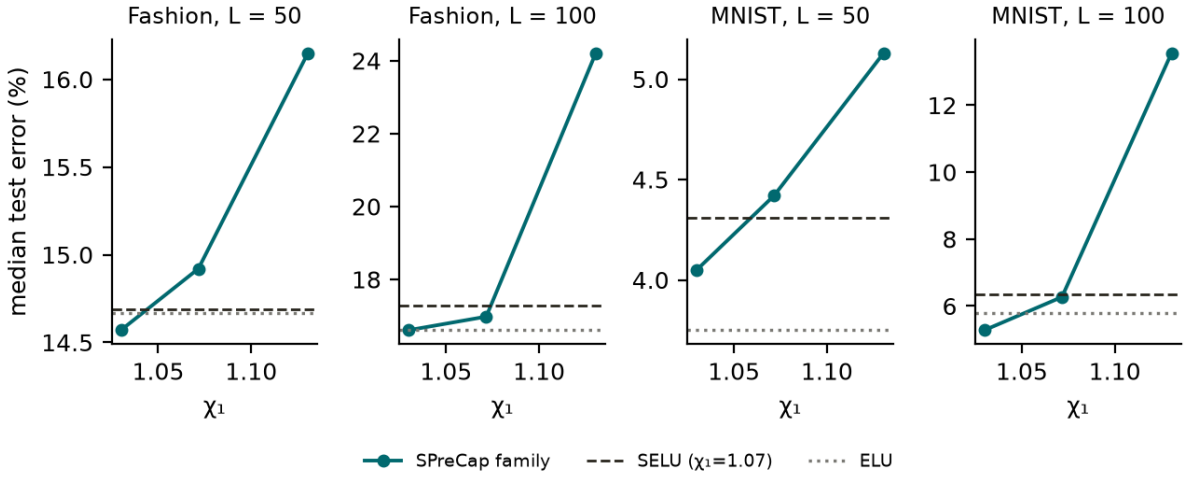


Figure 3. Median test error of the SPreCap family against χ_1, with SELU and ELU for reference.

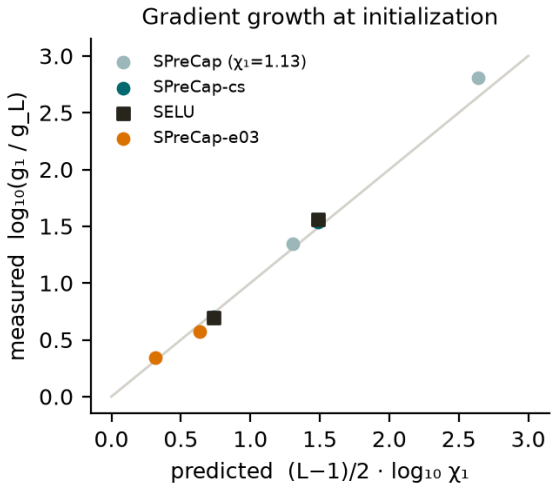


Figure 4. Gradient growth from the last to the first layer at initialization, at depth 50 (lower point) and 100 (upper point), against the prediction $(L-1)/2 \cdot \log_{10} \chi_1$. The line marks equality.

Learning rate. PreCap is not more tolerant of large learning rates (Figure 5). At depth 50 and learning rate $5 \cdot 10^{-4}$, SELU and ELU still reach 59–72 % validation accuracy, all PreCap variants 10–15 %. At depth 100 only the original SPreCap still learns at 10^{-4}. This shows a trade-off: larger χ_1 tolerates larger steps but reaches a worse optimum. The learnable clip s moved to a median of about 1.26–1.34 and helped only on MNIST at depth 50.

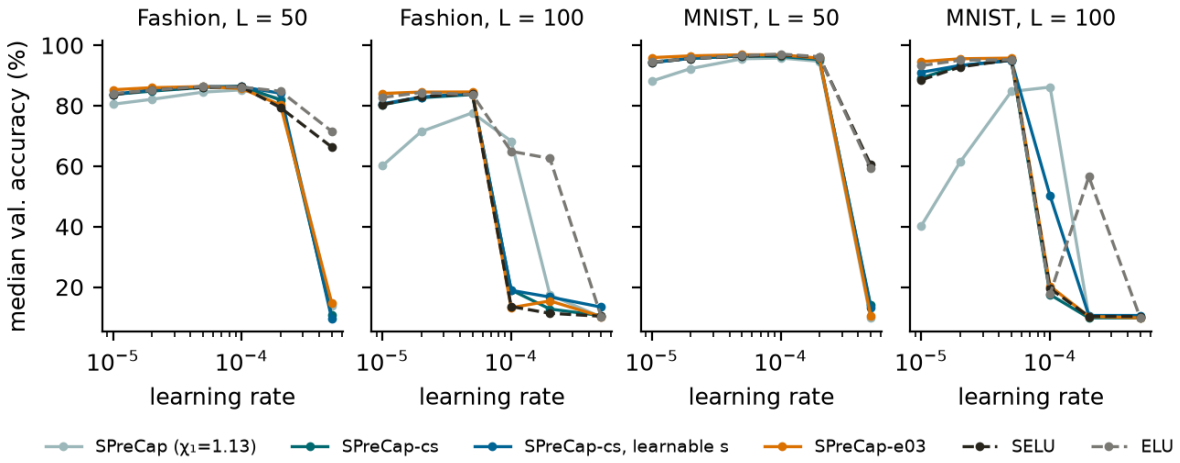


Figure 5. Median validation accuracy over the 4 tuning seeds for every learning rate in the grid.

History and negative results. The project began with 1D, 2D and 3D regression tasks (more than 19,000 runs). At a fixed learning rate of 10^{-3}, PreCap was far better than SiLU and GELU in residual networks. This advantage vanished once every activation received its own tuned learning rate (0 of 30 tests better, 2 worse), and a 3D transfer study (7200 runs) confirmed this. The explanation is scale: PreCap has slope 1 at the origin, SiLU and GELU 0.5. One robust property remained. The bound keeps undamped 128-layer residual networks trainable where SiLU and GELU diverge. In the first deep-MLP study (Phase 3a, 2430 runs) PreCap0 failed to learn at depth 100. In Phase 3a-2 (1092 runs) the original SPreCap was significantly worse than SELU in 4 of 4 cells. Its larger χ_1 (1.13 vs. 1.07) inflated input-side gradients by about 600× at depth 100, compared with 35× for SELU. This diagnosis motivated the χ_1 family above. All of these studies, including the failed ones, are documented in the repository.

5.2 Depth calibration (study A, 882 runs)

Both kL1 and kL2 are better than SELU in all 8 primary cells (depth 100 and 200, MNIST and FashionMNIST), each with 15 of 15 seeds and the smallest possible Holm-adjusted p (0.0005) (Table 5, Figure 7).

Cell (data, depth)	SELU	SPreCap-cs	SPreCap-e03	kL1	kL2
MNIST, 100	6.46	6.31	5.06	**4.56**	5.04
FashionMNIST, 100	16.64	16.95	16.64	**15.51**	15.62
MNIST, 200	39.53	40.04	8.73	5.97	**5.44**
FashionMNIST, 200	32.19	32.99	19.44	17.05	**16.53**

Table 5. Median test error (%) over 15 confirmation seeds, 1720 steps. Best per row in bold.

At depth 100 the advantage is moderate (−1.76 and −1.36 pp for kL1). At depth 200, SELU and SPreCap-cs do not finish training within 1720 steps (training loss about 0.9–1.3), so the large differences of up to −34 pp mainly measure training speed under a fixed budget, not the reachable final accuracy. Against the fixed $\chi_1 = 1.03$ of SPreCap-e03, the depth-dependent χ_1 helps mainly at depth 200. At depth 50 the depth family shows no consistent advantage. The measured gradient growth at initialization ranks the activations exactly as predicted by $(L-1)/2 \cdot \log_{10} \chi_1$ in all 6 cells.

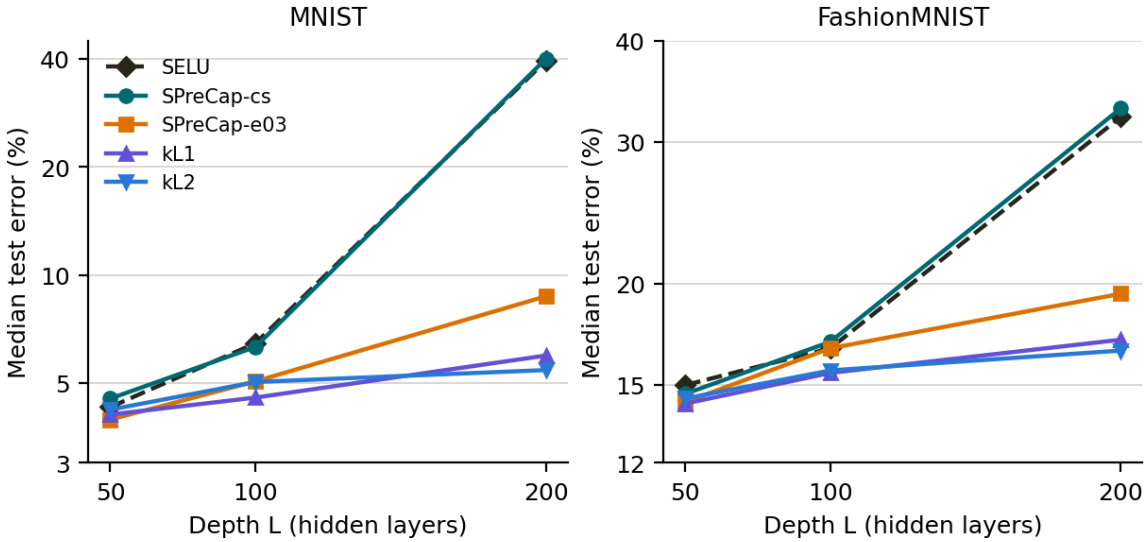


Figure 7. Median test error against depth (study A, 15 confirmation seeds, 1720 steps, logarithmic axis). SELU and SPreCap-cs fail to finish training at depth 200 within this budget; the depth family kL1/kL2 degrades least.

With the longer budget of 6880 steps (study A2), kL2 stays better than SELU at depth 100 (−1.08 and −2.87 pp, 10 of 10 seeds). At depth 200 the result is inconclusive, because SELU then trains through in some seeds (median differences −4.56 and −15.55 pp, 8 of 10 seeds, Holm p = 0.55).

Price. The depth family gives up the narrow range. The median maximum activation grows from 3.3 (cs) to 7.0–11.1 at depth 100 and 10.3–14.2 at depth 200. The median 4-bit loss (w8a4_best) at depth 100 rises to 18–31 pp, compared with about 5 pp for SELU. At 8 bits there is no difference.

5.3 Comparison with TAT (studies A2 and A3, 478 runs)

Own recipes (A2). kL2 with Gaussian initialization and without input normalization, as in study A, is worse than TAT in all 8 primary cells: by 1.5 to 2.5 pp at 1720 steps (0 of 15 seeds better in every cell) and by 0.4 to 1.9 pp at 6880 steps. TAT is also better than SELU in all 8 cells.

Same recipe (A3). A3 changes one thing, the recipe: kL2 is trained with TAT's recipe (SUO initialization and per-example input normalization) and a newly tuned learning rate. The difference to TAT then largely disappears (Table 6). The overall verdict under the preregistered rule is mixed: 2 cells better, 1 worse, 5 inconclusive. The recipe alone improves kL2 by 1.3 to 2.9 pp at 1720 steps, about as much as the gap to TAT in A2.

Budget	Cell	kL2 (Gauss)	kL2r	TAT	kL2r − TAT	W / L	Holm p	Verdict
1720	MNIST, 100	5.04	3.70	3.52	+0.12	5 / 10	1.00	inconclusive
1720	Fashion, 100	15.62	13.28	13.58	−0.35	12 / 2	0.046	better
1720	MNIST, 200	5.44	3.80	3.86	−0.12	8 / 7	1.00	inconclusive
1720	Fashion, 200	16.53	13.55	14.04	−0.77	14 / 1	0.0015	better
6880	MNIST, 100	3.21	2.84	2.84	−0.08	6 / 4	1.00	inconclusive
6880	Fashion, 100	13.08	12.53	12.07	+0.42	1 / 9	0.027	worse
6880	MNIST, 200	4.71	3.15	3.01	+0.16	3 / 7	0.80	inconclusive
6880	Fashion, 200	14.73	12.93	12.98	−0.19	7 / 3	1.00	inconclusive

Table 6. kL2 with TAT's recipe (kL2r) against TAT. Median test error (%) and median paired difference (pp); W / L = seeds in which kL2r wins / loses. kL2 (Gauss) is the recipe of study A, shown for reference.

With the longer budget, 3 of 4 median test errors of kL2r are equal to or slightly above those of TAT. This is not significant, but it suggests that TAT may gain with more training. We therefore claim only that kL2 is on par with TAT in fp32 under the same recipe, not that it is better. At depth 200 the selected learning rate of kL2r (and of TAT on FashionMNIST) lies at the lower edge of the grid.

5.4 Robustness to 4-bit activation quantization (studies Q1, Q1b, Q2, Q3, 818 runs)

Q1 and Q1b must be read together. Q1 alone would strongly overstate the effect.

Against SELU (Q1 and Q1b). With max calibration (Q1), SPreCap-cs loses far less accuracy than SELU at w8a4 in all 4 cells (−6.0 to −19.3 pp, 15 of 15 seeds each). At w8a8, all activations lose practically nothing ($|\Delta| \leq$ 0.16 pp). Max calibration, however, penalizes unbounded functions. With the best of four calibrations per run (Q1b), the advantage shrinks to 1.3–2.4 pp (Table 7, Figure 8). It remains uniform at depth 100 (15 of 15 seeds in both datasets), and it reverses on MNIST at depth 50, where SELU loses less. The bound helps the activations, not the weights: with 4-bit weights (w4a8), cs is not more robust than SELU. Against ELU, cs is more robust in all cells and under every calibration.

Cell (data, depth)	Median difference (pp)	W / L	Holm p	Verdict
MNIST, 50	+0.71	1 / 14	0.0006	less robust
FashionMNIST, 50	−1.33	15 / 0	0.0002	more robust
MNIST, 100	−2.35	15 / 0	0.0002	more robust
FashionMNIST, 100	−1.92	15 / 0	0.0002	more robust

Table 7. Q1b, w8a4_best: loss of accuracy through quantization, difference cs − SELU. Negative means cs loses less.

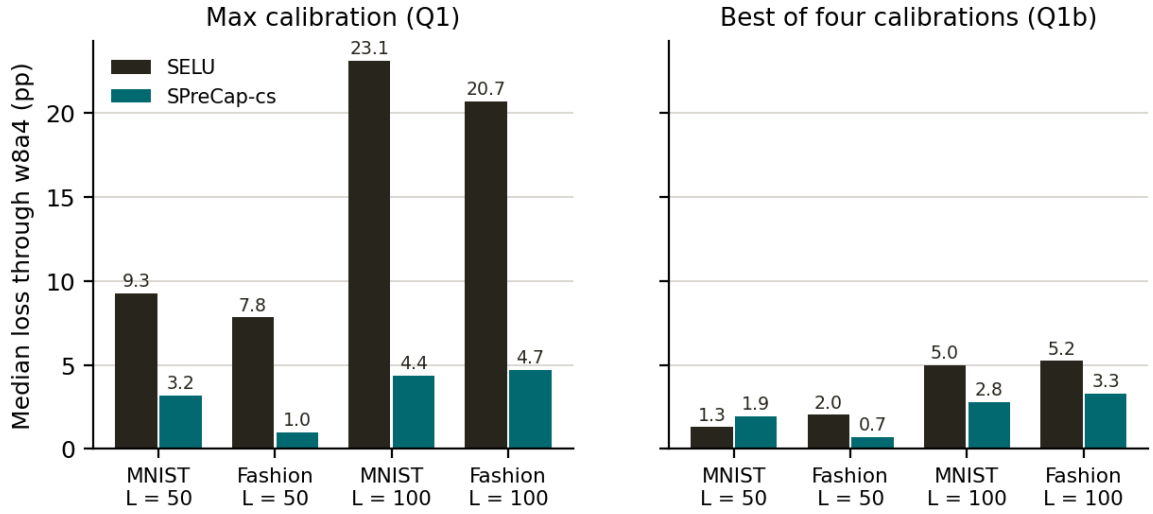


Figure 8. Median loss of test accuracy through w8a4 quantization for SELU and SPreCap-cs (15 seeds per bar). Left: max calibration (Q1). Right: best of max, 99.9th and 99.99th percentile and MSE clipping per run (Q1b, which retrained the Q1 networks bit-identically).

Against TAT (Q2 and Q3). TAT is the stronger method in fp32, so the quantization question is whether the bound changes the ranking at 4 bits. It does (Table 8, Figure 9). With symmetric per-tensor quantization (Q2), cs has a lower test error than TAT at w8a4_best in 4 of 4 cells. TAT loses 4 to 14 pp through 4-bit quantization, cs 1 to 3.5 pp. At 4 bits TAT is even worse than SELU in all 4 cells. Leaky ReLU is strongly asymmetric, so a symmetric grid wastes part of its 16 levels. With asymmetric per-tensor quantization (Q3), the 4-bit loss of TAT roughly halves. The advantage of cs shrinks to 1.1–2.2 pp but stays significant in 3 of 4 cells; on MNIST at depth 50 there is no difference. Under max calibration, TAT falls far behind (18–40 % test error), cs hardly (5.9–20.2 %), so cs does not depend on careful calibration.

Cell (data, depth)	fp32	w8a4_best, symmetric (Q2)	w8a4_best, asymmetric (Q3)
MNIST, 50	+0.81	−1.37 (13 / 2)	+0.34 (7 / 8), inconclusive
FashionMNIST, 50	+1.15	−7.71 (15 / 0)	−2.16 (14 / 1)
MNIST, 100	+2.68	−6.53 (15 / 0)	−1.08 (13 / 2)
FashionMNIST, 100	+3.93	−6.87 (15 / 0)	−1.96 (15 / 0)

Table 8. SPreCap-cs against TAT: median paired difference cs − TAT in test error (pp), W / L in parentheses. All shown tests have Holm $p \leq 0.005$ except MNIST 50 asymmetric ($p = 0.93$).

Medians of the test error at depth 100 make the trade-off concrete. On MNIST, TAT reaches 3.52 % in fp32 and cs 6.21 %; at asymmetric 4 bits, TAT reaches 8.94 % and cs 7.65 %. On FashionMNIST, the values are 13.58 % against 17.25 % in fp32 and 21.13 % against 19.24 % at asymmetric 4 bits.

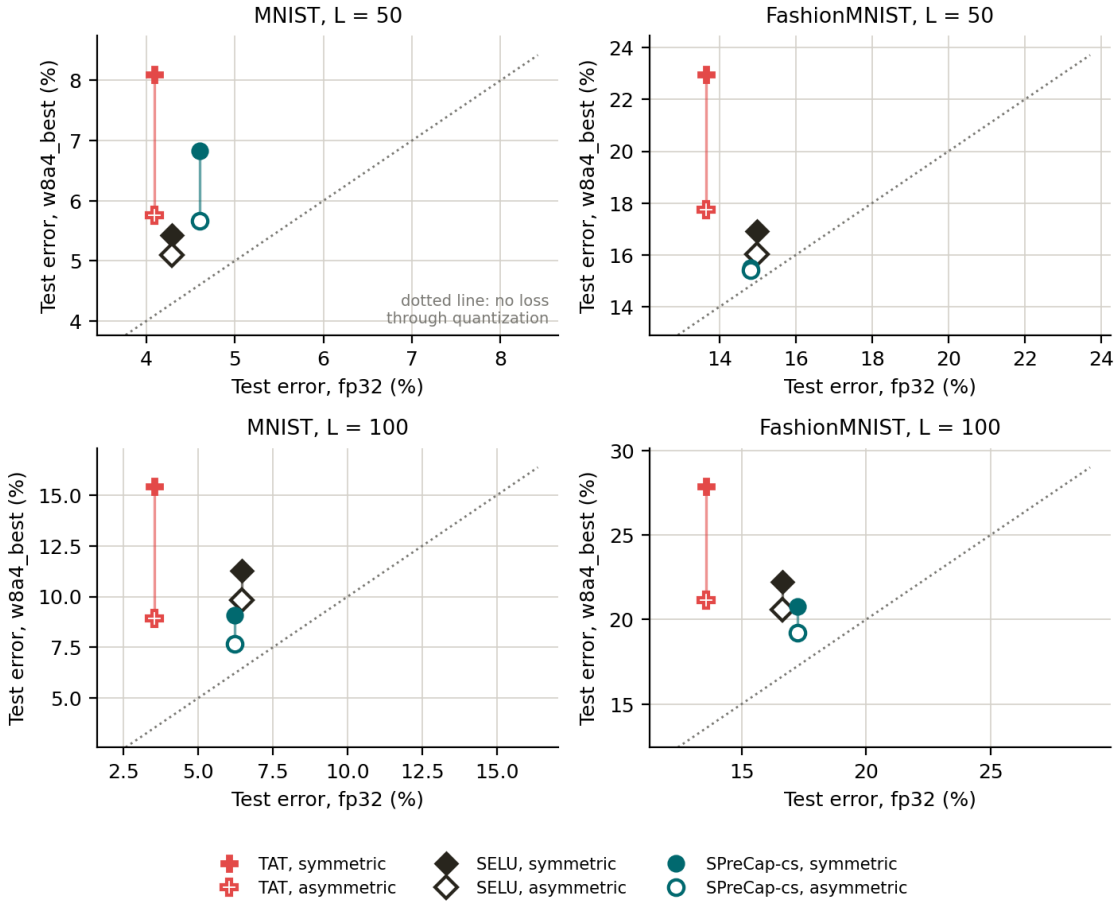


Figure 9. Median test error in fp32 (horizontal) against w8a4_best (vertical), with symmetric (filled) and asymmetric (open) per-tensor activation quantization (study Q3, 15 seeds). The dotted line marks no loss through quantization. TAT is best in fp32 but loses most at 4 bits.

5.5 Summary of version 0.2

Claim	Evidence	Status
kL1/kL2 better than SELU at depth 100/200, 1720 steps	A, 8/8 cells, 15/15 seeds	confirmed
kL2 better than SELU at 6880 steps, depth 100	A2, secondary	confirmed; depth 200 inconclusive
TAT better than kL2 with Gaussian initialization	A2, 8/8 cells	confirmed
kL2 on par with TAT in fp32 under TAT's recipe	A3, 2 better, 1 worse, 5 inconclusive	mixed, as preregistered
cs more robust than SELU at 4 bits, fair calibration	Q1 and Q1b	depth 100 confirmed; depth 50 mixed
cs more robust than TAT at 4 bits	Q2 (symmetric, 4/4), Q3 (asymmetric, 3/4)	confirmed for per-tensor PTQ; cs worse in fp32 (4/4)
Depth calibration costs 4-bit robustness	A, descriptive	observed

Table 9. Claims of version 0.2 and their evidence.

6 Kernels and package

An unfused PreCap needs about ten element-wise operations per direction. We provide fused forward and backward kernels: CUDA C++ compiled at run time through `torch.cuda.jiterator` (no CUDA toolkit required) and a Metal kernel through `mx.fast.metal_kernel` with a custom VJP for MLX. On an RTX 4070 all 17 checks pass (values and gradients in fp32/fp64, gradcheck, edge values up to $\pm 10^4$, a 20-layer MLP trained with kernel vs. reference), with a maximum relative fp32 error of $1.1 \cdot 10^{-6}$. A second, extended run on the same GPU passes 14 further checks: fp16 and bf16 values and gradients for all four presets (maximum relative error $3.8 \cdot 10^{-4}$ in fp16 and $2.9 \cdot 10^{-3}$ in bf16, i.e. at the rounding level of the format), NaN propagation, ±Inf mapped to the two bounds, non-contiguous tensors, and identical results of the package module and the reference. Speed varies between runs: forward plus backward over 16M fp32 elements took 0.91 ms vs. 0.89 ms for native SELU in the first run (Figure 6) and 1.06 ms vs. 0.73 ms in the second, where ReLU, SiLU, GELU and ELU also took 0.73 ms, so the fused kernel costs up to about 1.5× a native activation. In fp16 the kernel is slower (1.61 ms vs. 0.34 ms for SELU) because inputs are converted to fp32 and back in separate passes. On an M1 Pro all 12 checks pass. The package installs with

```
pip install "precap[torch] @ git+https://github.com/hermi-labs/precap-activation"
```

and exposes `from precap import PreCap` (the CUDA kernel is used automatically on CUDA tensors) and `from precap.mlx import PreCap`.

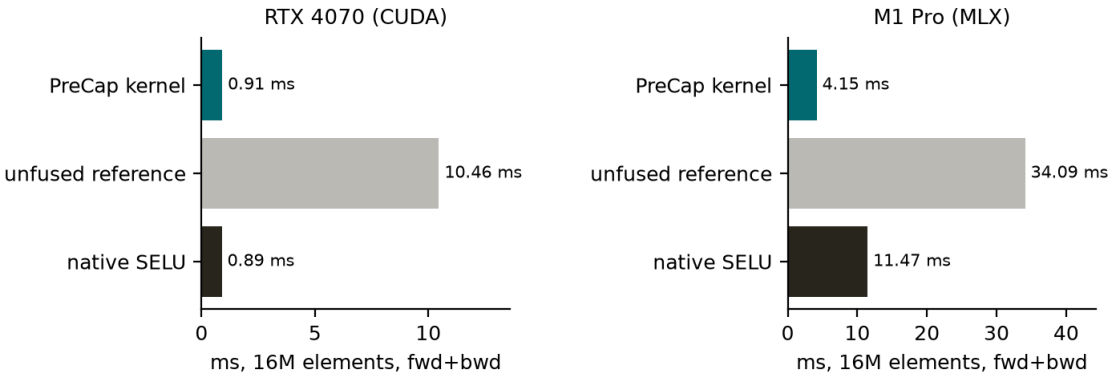


Figure 6. Time for forward plus backward over 16M float32 elements. The MLX measurement ran alongside a training job and is likely pessimistic.

Version 0.2.0 of the repository adds the constants of Table 4 and the code, preregistrations and raw results of all v0.2 studies.

7 Limitations

- Only fully connected networks, one width (256), MNIST and FashionMNIST. No CNNs, Transformers or larger datasets.

- Short budgets (1720 and 6880 steps). At depth 200 the comparison with SELU mainly measures training speed.
- TAT was trained with Adam instead of K-FAC. With K-FAC it may be stronger in fp32. At depth 200 the learning rates of kL2r and of TAT on FashionMNIST lie at the lower edge of the grid.
- Quantization is simulated post-training quantization, per tensor for activations. Per-channel activation quantization and quantization-aware training were not tested, and no real int4 hardware was used.
- The fp32 advantage of the depth family (kL2) and the 4-bit robustness (cs) belong to different members of the family. No single variant offers both.
- The advantage of SPreCap-e03 over SELU from version 0.1 rests on one cell of a secondary family. The depth-dependent linearization is a known principle (DKS, TAT, shaped activations), and so is the depth bound $\varepsilon < 1/L$ for SELU [17]; we do not claim either as new.
- The CUDA kernel has no native fp16/bf16 path and is about 4.7× slower than native SELU in fp16.
- The work was done by a single independent researcher on consumer hardware, with AI assistance for code, analysis and writing (see acknowledgments).

8 Conclusion

The bounded PreCap family is not a general improvement over the state of the art. Its depth-calibrated member trains 100- and 200-layer MLPs without normalization clearly better than SELU and reaches the level of TAT in fp32 once it uses the same recipe. Its bounded member, SPreCap-cs, is more robust than SELU at depth 100 and more robust than TAT when activations are quantized to 4 bits after training, at a cost in fp32 accuracy. The bound is a measurable advantage only at low activation precision, and the advantage shrinks with fair calibration and asymmetric quantization. Whether it carries over to convolutional networks and Transformers, where 4-bit inference matters most, is open.

Acknowledgments. Idea, function and all experiments by the author. Implementation, statistical analysis, literature research and writing were supported by Perplexity Computer with Claude models (version 0.1) and Claude Code (version 0.2), both by Anthropic. See `docs/Danksagung_und_Werkzeuge.md`.

Reproducibility. For every study of version 0.2, the repository contains the preregistration, the frozen scripts with SHA-256 hashes and freeze times, raw results per run, the analysis script and output, and the report. The preregistrations are published unchanged; later corrections are noted in the reports, not in the frozen files. Figure and paper sources are in the repository under Apache-2.0.

References

[1] Klambauer, Unterthiner, Mayr, Hochreiter. Self-Normalizing Neural Networks. NeurIPS 2017. https://arxiv.org/abs/1706.02515

[2] Chen, Zhao, Deng, Ding, Wen, Li, Xie. Large-scale self-normalizing neural networks (bSELU). Journal of Automation and Intelligence 3, 101–110, 2024. https://www.sciopen.com/article/10.1016/j.jai.2024.05.001

[3] Hayou, Doucet, Rousseau. On the Impact of the Activation Function on Deep Neural Networks Training. ICML 2019. https://arxiv.org/abs/1902.06853

[4] Lu, Gould, Ajanthan. Bidirectionally Self-Normalizing Neural Networks. 2020/2021. https://arxiv.org/abs/2006.12169

[5] Zaied. SaluNet: Enabling Total Plasticity in Normalization-Free Networks. 2026. https://arxiv.org/abs/2606.02927

[6] Zhu, Chen, He, LeCun, Liu. Transformers without Normalization (DyT). CVPR 2025. https://arxiv.org/abs/2503.10622

[7] Chen, Lu, Zhu, Sun, Liu. Stronger Normalization-Free Transformers (Derf). 2025. https://arxiv.org/abs/2512.10938

[8] Clevert, Unterthiner, Hochreiter. Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs). ICLR 2016. https://arxiv.org/abs/1511.07289

[9] Bukva, de Gier, Grosvenor, Jefferson, Schalm, Schwander. Criticality versus uniformity in deep neural networks. 2023. https://arxiv.org/abs/2304.04784

[10] Liu, Di. TanhExp: A Smooth Activation Function with High Convergence Speed for Lightweight Neural Networks. IET Computer Vision. https://arxiv.org/abs/2003.09855

[11] Kunc, Kléma. Three Decades of Activations: A Comprehensive Survey of 400 Activation Functions for Neural Networks. 2024. https://arxiv.org/abs/2402.09092

[12] Zhang, Botev, Martens. Deep Learning without Shortcuts: Shaping the Kernel with Tailored Rectifiers. ICLR 2022. https://arxiv.org/abs/2203.08120

[13] Martens, Ballard, Desjardins, Swirszcz, Dalibard, Sohl-Dickstein, Schoenholz. Rapid training of deep neural networks without skip connections or normalization layers using Deep Kernel Shaping. 2021. https://arxiv.org/abs/2110.01765

[14] Li, Nica, Roy. The Neural Covariance SDE: Shaped Infinite Depth-and-Width Networks at Initialization. NeurIPS 2022. https://proceedings.neurips.cc/paper_files/paper/2022/hash/45fc4a0da7e7f6fbabaabe2d20a441d1-Abstract-Conference.html

[15] Nagel, Fournarakis, Amjad, Bondarenko, van Baalen, Blankevoort. A White Paper on Neural Network Quantization. 2021. https://arxiv.org/abs/2106.08295

[16] Choi, Wang, Venkataramani, Chuang, Srinivasan, Gopalakrishnan. PACT: Parameterized Clipping Activation for Quantized Neural Networks. 2018. https://arxiv.org/abs/1805.06085

[17] Chen, Deng, Wang, Li, Xie. A Comprehensive and Modularized Statistical Framework for Gradient Norm Equality in Deep Neural Networks. IEEE TPAMI 2020. https://arxiv.org/abs/2001.00254